

Pembangunan SIG Web

WebGIS vs Desktop GIS, Maps API, & Publikasi

Program Sarjana Terapan Teknologi Survei dan Pemetaan Dasar
Departemen Teknologi Kebumian
Sekolah Vokasi, UGM

Ismail Sunni | Geospatial Software Engineer | Camptocamp DE

Presentasi Ini

<https://github.com/ismailsunni/web-gis-2026>



Tujuan Pembelajaran

Setelah sesi ini, mahasiswa mampu:

- ✓ Membandingkan **WebGIS vs Desktop GIS** secara mendalam
- ✓ Menjelaskan **arsitektur & komponen** sistem WebGIS
- ✓ Membandingkan **Maps API** (Leaflet, OpenLayers, Google Maps) untuk kasus nyata
- ✓ Mendemonstrasikan **OpenLayers** untuk WebGIS profesional
- ✓ Mempublikasi peta interaktif ke **GitHub Pages**

Recap: Minggu 5

Yang sudah dipelajari:

- JavaScript objects, methods, properties
- Pengenalan **Leaflet** & **OpenLayers** sebagai library
- Client-side scripting dasar

Hari ini: Naik level — dari *library* ke *sistem*

Outline

1. **WebGIS vs Desktop GIS** — Perbandingan mendalam
2. **Arsitektur WebGIS** — Stack & komponen
3. **Deep Dive: Maps API** — Leaflet vs OpenLayers vs Google Maps
4. **Hands-on: OpenLayers** — WebGIS profesional dari nol
5. **Strategi Publikasi** — Deploy & distribusi
6. **Studi Kasus** — WebGIS di dunia nyata
7. **Refleksi & Tugas**

1 WebGIS vs Desktop GIS

Lebih dari Sekadar "Bisa Online"

Desktop GIS: Kekuatan Utama

QGIS / ArcGIS Desktop / GRASS

- Analisis spasial lengkap — buffer, overlay, network analysis
- Editing geometri presisi tinggi
- Cartographic production — peta cetak berkualitas
- Processing pipeline — batch geoprocessing
- Data format beragam — Shapefile, GeoPackage, GDB, raster

Desktop GIS = laboratorium analisis spasial

Web GIS: Kekuatan Utama

Google Maps / Mapbox / Custom WebGIS

- Zero install — akses via URL
- Multi-platform — desktop, tablet, HP
- Kolaborasi real-time — banyak user bersamaan
- Integrasi — embed di website, dashboard, CMS
- Update sentral — data berubah, semua user dapat

Web GIS = media komunikasi & distribusi spasial

Perbandingan Mendalam

Aspek	Desktop GIS	Web GIS
Analisis	Lengkap (500+ tools)	Terbatas (client-side)
Data size	GB–TB lokal	MB–ratusan MB streaming
CRS	Semua CRS	Umumnya Web Mercator
Rendering	GPU lokal	Browser (Canvas/WebGL)
Offline	✅ Penuh	⚠️ Terbatas
Kolaborasi	❌ File-based	✅ Real-time
Distribusi	Email/FTP file	Share URL
Maintenance	Update per mesin	Update 1× di server

Hybrid Approach: Best of Both Worlds



Workflow modern: Analisis di desktop → Publikasi di web

Kapan Pakai Yang Mana?



Desktop GIS:

- Analisis kompleks (interpolasi, network, 3D)
- Digitasi presisi
- Peta cetak / kartografi



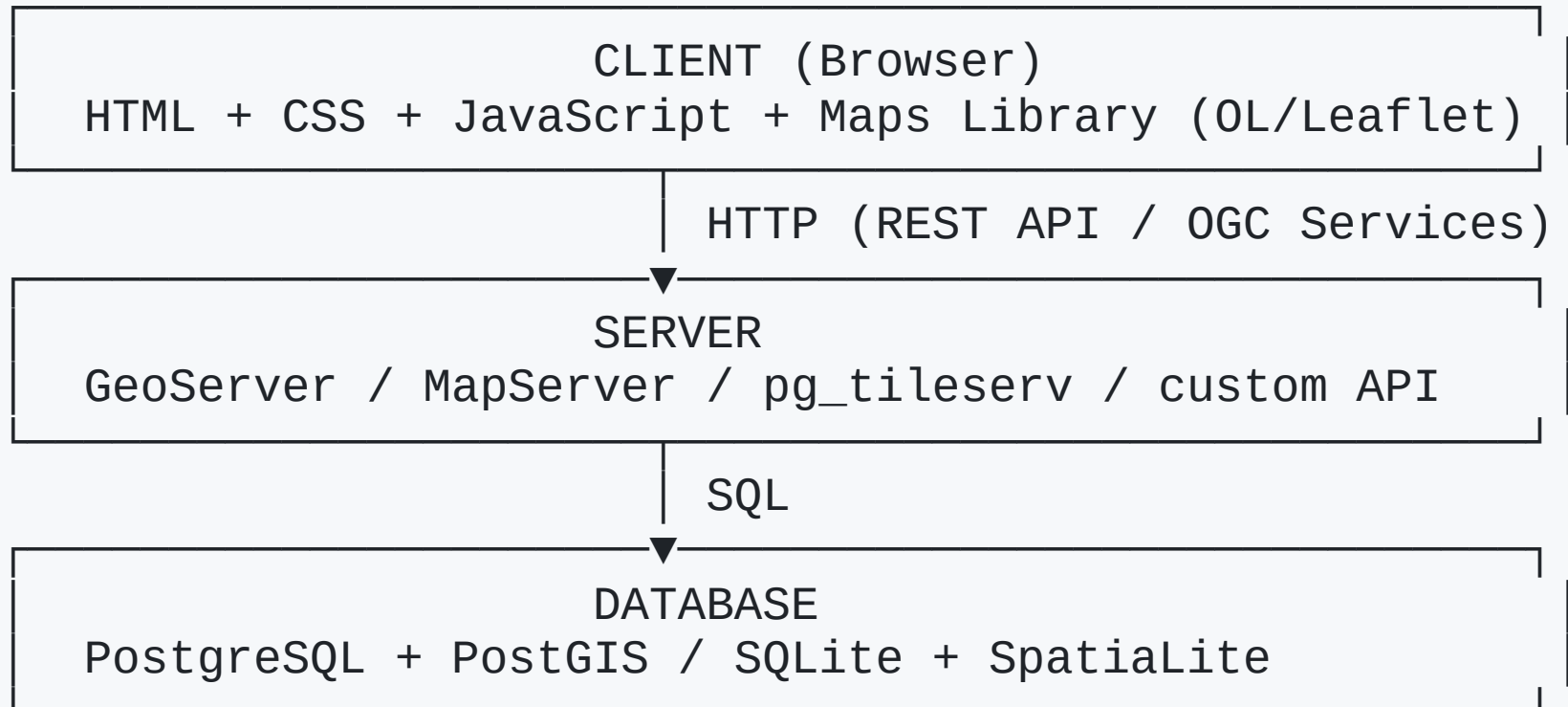
Web GIS:

- Dashboard monitoring
- Informasi publik
- Kolaborasi tim
- Aplikasi mobile/field

2 **Arsitektur WebGIS**

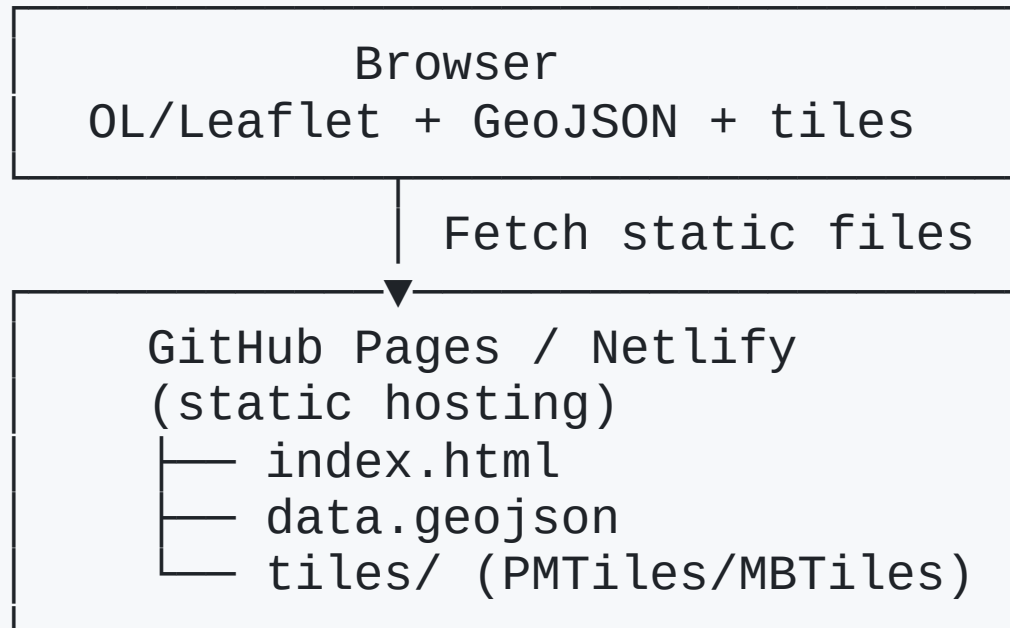
Dari Database ke Browser

Full Stack WebGIS



Alternatif: Serverless WebGIS

Tidak semua WebGIS butuh backend!



Gratis, cepat, tanpa server!

Format Data di WebGIS

Format	Tipe	Cocok untuk
GeoJSON	Vektor	Fitur kecil-sedang (<10 MB)
TopoJSON	Vektor (compressed)	Fitur besar, batas wilayah
XYZ Tiles	Raster	Base map (OSM, satellite)
PMTiles	Vektor tiles	Offline / static hosting
WMS	Raster	Server-rendered map image
WFS	Vektor	Server feature access
COG	Raster (cloud)	Citra satelit besar

3 Deep Dive: Maps API

Perbandingan untuk Kasus Nyata

Recap: 3 Maps API Utama

Dari minggu 5, kalian sudah kenal:

- **Leaflet** 🌿 — ringan, simpel
- **OpenLayers** 🌐 — lengkap, GIS-grade
- **Google Maps** 🗺️ — familiar, proprietary

Sekarang kita bandingkan untuk **kasus nyata**.

Kasus 1: Peta Wisata Desa

Kebutuhan: Marker lokasi wisata, foto popup, mobile-friendly

Kriteria	Leaflet	OpenLayers	Google Maps
Complexity	★ Simpel	★★ Overkill	★★ Perlu API key
Mobile	✓ Baik	✓ Baik	✓ Sangat baik
Biaya	Gratis	Gratis	Berbayar*

Pilihan: Leaflet ← paling efisien untuk kasus ini

Kasus 2: Monitoring Bencana (BPBD)

Kebutuhan: WMS dari GeoServer, real-time update, multi-layer, CRS lokal

Kriteria	Leaflet	OpenLayers	Google Maps
WMS/WFS	⚠️ Plugin	✅ Native	❌ Tidak ada
CRS lokal	❌ Tanpa plugin	✅ Semua CRS	❌ Hanya EPSG:3857
Layer control	Dasar	Lengkap	Dasar

Pilihan: OpenLayers ← mendukung OGC standards natively

Kasus 3: Aplikasi Ride-hailing

Kebutuhan: Routing, geocoding, traffic, familiar UI

Kriteria	Leaflet	OpenLayers	Google Maps
Routing	Plugin (OSRM)	Plugin	✓ Built-in
Geocoding	Plugin (Nominatim)	Plugin	✓ Built-in
Traffic	✗	✗	✓ Built-in

Pilihan: Google Maps ← kalau budget ada & butuh data Google

Keputusan Cepat

Proyek sederhana, gratis?

→ Leaflet

Butuh WMS/WFS/OGC, multi-CRS?

→ OpenLayers

Butuh routing/geocoding Google?

→ Google Maps API

Butuh visualisasi 3D, vector tiles?

→ Mapbox GL JS / MapLibre GL JS

4 Hands-on: OpenLayers

WebGIS Profesional dari Nol

Mengapa OpenLayers Hari Ini?

Minggu 5: sudah coba Leaflet

Hari ini: OpenLayers — library yang dipakai di:

-  GeoNode (BNPB, BIG)
-  map.geo.admin.ch (Swiss topo)
-  IGN Géoportail (Prancis)
-  Banyak portal pemerintah

Kalau mau kerja di enterprise GIS → harus tahu OpenLayers

Step 1: HTML + OpenLayers via CDN

```
<!DOCTYPE html>
<html>
<head>
  <title>WebGIS Yogyakarta - OpenLayers</title>
  <link rel="stylesheet"
    href="https://cdn.jsdelivr.net/npm/ol@10/dist/ol.css" />
  <style>
    #map { width: 100%; height: 100vh; }
  </style>
</head>
<body>
  <div id="map"></div>
  <script src="https://cdn.jsdelivr.net/npm/ol@10/dist/ol.js"></script>
  <script src="app.js"></script>
</body>
</html>
```


Step 2: Inisialisasi Peta

```
const map = new ol.Map({  
  target: 'map',  
  layers: [  
    new ol.layer.Tile({  
      source: new ol.source.OSM()  
    })  
  ],  
  view: new ol.View({  
    center: ol.proj.fromLonLat([110.3695, -7.7956]),  
    zoom: 14  
  })  
});
```

Perhatikan: `ol.proj.fromLonLat()` — OL menggunakan proyeksi EPSG:3857

Step 3: Multiple Base Maps

```
const osmSource = new ol.source.OSM();  
const cartoSource = new ol.source.XYZ({  
  url: 'https://basemaps.cartocdn.com/light_all/{z}/{x}/{y}.png'  
});  
  
const baseLayer = new ol.layer.Tile({ source: osmSource });  
// Switch: baseLayer.setSource(cartoSource)
```

Berbeda dari Leaflet yang membuat layer baru —
OpenLayers bisa **ganti source** pada layer yang sama

Step 4: Markers via Vector Layer

```
const markerSource = new ol.source.Vector();

function addMarker(lon, lat, name) {
  const feature = new ol.Feature({
    geometry: new ol.geom.Point(ol.proj.fromLonLat([lon, lat])),
    name: name
  });
  feature.setStyle(new ol.style.Style({
    image: new ol.style.Circle({
      radius: 8,
      fill: new ol.style.Fill({ color: '#e63946' }),
      stroke: new ol.style.Stroke({ color: '#fff', width: 2 })
    })
  }));
  markerSource.addFeature(feature);
}

const markerLayer = new ol.layer.Vector({ source: markerSource });
map.addLayer(markerLayer);
```

Step 5: Popup via Overlay

```
const popup = document.createElement('div');
popup.className = 'ol-popup';
document.body.appendChild(popup);

const overlay = new ol.Overlay({ element: popup, positioning: 'bottom-center' });
map.addOverlay(overlay);

map.on('click', (e) => {
  const feature = map.forEachFeatureAtPixel(e.pixel, f => f);
  if (feature) {
    popup.innerHTML = `${feature.get('name')}`;
    overlay.setPosition(e.coordinate);
  } else {
    overlay.setPosition(undefined);
  }
});
```

Step 6: GeoJSON Layer

```
const geojsonLayer = new ol.layer.Vector({
  source: new ol.source.Vector({
    url: 'data.geojson',          // file lokal atau URL
    format: new ol.format.GeoJSON()
  }),
  style: new ol.style.Style({
    stroke: new ol.style.Stroke({ color: '#2563eb', width: 2 }),
    fill: new ol.style.Fill({ color: 'rgba(37,99,235,0.15)' })
  })
});
map.addLayer(geojsonLayer);
```

Leaflet vs OpenLayers: Side-by-Side

Aksi	Leaflet	OpenLayers
Buat peta	<code>L.map('map')</code>	<code>new ol.Map({target:'map'})</code>
Set view	<code>.setView([-7.79, 110.36], 13)</code>	<code>view: new ol.View({center:..., zoom:13})</code>
Tambah tile	<code>L.tileLayer(url).addTo(map)</code>	<code>new ol.layer.Tile({source:...})</code>
Marker	<code>L.marker([lat, lng])</code>	<code>new ol.Feature({geometry: new ol.geom.Point(...)})</code>
Popup	<code>.bindPopup(html)</code>	<code>new ol.Overlay({element:...})</code>
		<code>new ol.source.Vector({format:</code>

5 Strategi Publikasi

Dari Laptop ke Internet

3 Tingkat Publikasi

Level 1: Static (gratis)

- GitHub Pages, Netlify, Vercel
- File HTML + JS + GeoJSON saja
-  Cocok untuk: portfolio, demo, peta sederhana

Level 2: Server GIS

- VPS + GeoServer + PostGIS
- WMS/WFS untuk data dinamis
-  Cocok untuk: instansi, monitoring

Level 3: Cloud Platform

Deploy ke GitHub Pages

Langkah Praktis:

```
# 1. Buat repo
git init
git add .
git commit -m "first webgis"

# 2. Push ke GitHub
git remote add origin https://github.com/user/repo.git
git push -u origin main

# 3. Aktifkan Pages
# Settings → Pages → Source: main branch → /root

# 4. Akses di:
# https://user.github.io/repo/
```

Tips Publikasi Profesional

1. **README.md** — jelaskan proyek, screenshot, link demo
2. **Responsive** — test di mobile (DevTools → toggle device)
3. **HTTPS** — GitHub Pages sudah otomatis
4. **Loading speed** — pakai CDN, compress gambar
5. **Metadata** — title, description, og:image
6. **Lisensi** — cantumkan sumber data & library

6 Studi Kasus

WebGIS di Dunia Nyata

Studi Kasus: map.geo.admin.ch

 **Geoportal nasional Swiss** — dibangun dengan OpenLayers

Fitur:

- 800+ layer data nasional
- 2D dan 3D view
- WMS/WMTS dari swisstopo
- Pencarian alamat & koordinat
- Cetak peta & share URL
- Open source!

 map.geo.admin.ch

Studi Kasus: Peta Interaktif Routing



Route Finder — contoh WebGIS dengan routing nyata

Fitur:

- Pencarian lokasi (Photon geocoding)
- Routing jalan nyata via pgRouting
- TSP solver (Held-Karp)
- Multiple kota (Yogyakarta & München)

 ismailsunni.id/map/route-finder

Dibangun dengan OpenLayers + Supabase + PostgreSQL

Studi Kasus: Peta dari Google Sheet

 Sheet Map — WebGIS tanpa backend

Fitur:

- Data dari Google Sheet publik
- Clustering otomatis
- Warna per kategori
- Zero server — 100% client-side

 ismailsunni.id/map/sheet-map

Cocok untuk: survei, data crowdsource, tugas mahasiswa

7 Refleksi & Tugas

Refleksi

- 🤔 Kapan sebaiknya pakai Desktop vs Web GIS?
- 🤔 Library mana paling cocok untuk proyek Anda?
- 🤔 Apakah semua WebGIS butuh server?
- 🤔 Bagaimana strategi publikasi yang paling efisien?

Key Takeaways

1. Desktop GIS = **analisis**, Web GIS = **komunikasi & distribusi**
2. **Hybrid workflow** paling efektif: analisis → publish → share
3. Leaflet → simpel, OpenLayers → GIS-grade, Google Maps → data Google
4. **Serverless WebGIS** bisa sangat powerful (GeoJSON + GitHub Pages)
5. **Publikasi = bagian dari proyek** — bukan tambahan

Tugas

Buat **WebGIS** interaktif dengan ketentuan:

1. Gunakan **OpenLayers** (bukan Leaflet — sudah minggu 5)
2. Tampilkan **minimal 5 lokasi** (marker dengan popup)
3. **Minimal 2 base map** (bisa switch)
4. Tambahkan **1 layer GeoJSON** (polygon atau polyline)
5. *(Bonus)* Tambahkan **interaksi**: click event, hover tooltip
6. *(Bonus)* Buat data dari **Google Sheet** → GeoJSON
7. **Deploy ke GitHub Pages**
8. Kumpulkan: **link repo + link demo**



Referensi

- [OpenLayers Quick Start](#)
- [OpenLayers Examples](#)
- [Leaflet vs OpenLayers Comparison](#)
- [GitHub Pages Docs](#)
- [GeoJSON.io](#) — buat GeoJSON interaktif
- [Map Collection](#) — contoh proyek
- [This presentation](#)

Eksplorasi dan bawa pertanyaan ke sesi berikutnya!